Thread Scaling and SIMD Vectorisation for Dense Matrix Multiplication: A Benchmarking Study

Alejandro Delgado Valera
BIG DATA – Individual Assignment 3
Grado en Ciencia e Ingeniería de Datos
Universidad de Las Palmas de Gran Canaria
https://github.com/aledelgadoo/individual-assignments-bigdata

October 27, 2026

Abstract

Question: how does the number of threads p affect a dense matrix multiplication on shared memory, when does it beat sequential execution, and what does SIMD vectorisation add on top of threads? We compare an equivalent sequential baseline with an OpenMP row-parallel implementation of the same validated `ikj` kernel (same `float64` data, inputs and arithmetic order). We measure $p \in \{1, 2, 4, 8, 12, 16\}$ on a 10-core/12-thread hybrid laptop CPU, study vectorisation at fixed p (scalar vs. compiler default vs. explicit pragma), and repeat the run confined to the four performance cores. Times are medians of five repetitions; the sequential reference is measured around every thread sweep to remove thermal drift. Parallelism pays off only from $n = 200$ upward and stops scaling around $p = 8$–12: the best speedup is $2.59\times$ at $n = 2000$, and efficiency never exceeds 0.77. With one thread the parallel code is 26 % slower at $n = 50$ (fork/join overhead). Running 16 threads on 12 execution contexts brings no gain and hurts small cases. Explicit SIMD is worth 1.4–$3.1\times$ at fixed p, and the evidence shows GCC did not vectorise the parallel loop by default. Confining the run to the performance cores is 12–14 % faster at $n = 2000$. All claims apply only to this kernel, machine and protocol.

1 Introduction and objective

Assignments 1 and 2 produced a validated single-threaded baseline: a dense $O(n^3)$ multiplication whose `ikj` loop order is the fastest local variant on this machine [1, 2]. This assignment studies how the same kernel behaves with several hardware threads, and how much of that behaviour comes from threads and how much from SIMD vectorisation.

Research question. How does the thread count p change the multiplication time $T_p(n)$? For which sizes is parallel execution faster than the sequential baseline? And what speed comes from vectorisation alone, with the thread count fixed?

Hypotheses (stated before measurement).

H1: Parallel speedup grows with n but stays far below p: fork/join costs dominate small multiplications, and at large n memory bandwidth and the mixed P/E cores limit scaling.

H2: The parallel code with one thread shows the framework overhead: $T_p(1) \geq T_{seq}(n)$, with the largest gap at small n.

H3: Running $p = 16$ threads on 12 hardware contexts (oversubscription) brings no extra gain and is most costly for small multiplications.

H4: At fixed p, the vectorised kernel is faster, with a smaller gain as n grows and the multiplication becomes memory-bound; and compiler-default vectorisation of the parallel loop must not be assumed—it has to be checked per binary.

2 Background

Speedup and efficiency. With $T_{seq}(n)$ the sequential time and $T_p(n)$ the parallel time for the same input, $S(p) = T_{seq}(n)/T_p(n)$ and $E(p) = S(p)/p \leq 1$ for a workload that does not change [6, 7]. A superlinear value ($S(p) > p$) would signal a measurement or cache effect and must be explained, not assumed [7].

OpenMP. A `parallel for` loop forks p threads, splits the outer index with a static schedule, and joins at the end [3]. Each thread writes disjoint rows of C, and the summation order over k is unchanged, so every p gives bitwise-identical results. The inner loop over j walks contiguous memory, which suits vectorisation.

Hybrid CPUs. The test laptop mixes performance cores (P-cores, two threads each) with efficiency cores (E-cores, one thread each). Splitting rows evenly gives every thread the same number of rows, but not the same cost: an E-core takes longer per row than a P-core. This limits $E(p)$ and makes thread placement part of the measurement [4].

SIMD. With `-march=native` the compiler targets AVX2 with FMA (32-byte vectors, i.e. four doubles per vector; AVX-512 is disabled by the toolchain). The `#pragma omp simd` requests vectorisation explicitly, and its effect must be measured at fixed p so it is not mixed with thread effects [5].

3 Methodology

3.1 Implementations

One C source (`src/kernels.c`) is compiled in four ways (Table 1). All binaries run the same `ikj` kernel on the same `float64` row-major matrices; the only differences are the compiler flags. The sequential and parallel kernels do the same operations in the same order (the parallel loop only distributes rows), so they are equivalent; the documented differences are the OpenMP runtime and the SIMD pragma.

Binary	Flags	Role
`matmul3_seq`	`-O2 -march=native`	sequential baseline
`matmul3_scalar`	`... -fopenmp -fno-tree-vectorize`	parallel, no SIMD (control)
`matmul3_auto`	`... -fopenmp`	parallel, compiler default
`matmul3_simd`	`... -fopenmp -DUSE_OMP_SIMD`	parallel, explicit pragma

Table 1: Four builds of one shared source (`-std=c11 -Wall -Wextra`, linked with `-lpsapi -lm`). Only `simd` adds `#pragma omp simd` on the inner loop.

Threads are controlled inside the process: `omp_set_dynamic(0)` and `omp_set_num_threads(`p`)` fix the team size before every timed run, and `schedule(static)` splits the rows evenly. No library with internal threads (no BLAS) is used, so the reported p is the real configuration. Affinity is set at process level with `SetProcessAffinityMask`: `all` = CPU0–11 and `pcore` = CPU0–3. Within the mask, placement is left to the Windows scheduler, because libgomp on Windows offers no reliable binding; this is recorded as a limitation.

3.2 Inputs

Dense matrices for $n \in \{50, 100, 200, 500, 1000, 2000\}$, generated as in Assignments 1–2 (seed 42, uniform in $[-1, 1]$, little-endian `<f8`, row-major) and reused byte for byte. A third buffer holds C, zeroed inside the timed kernel.

3.3 Correctness

The self-test (run on all four binaries; 31 checks for `seq`, 76 for the parallel builds; 0 failures) verifies known products, zero and identity matrices, and random cases against a long-double reference ($rtol = 10^{-10}$, $atol = 10^{-12}$). For the parallel builds it also checks **bitwise invariance** for $p \in \{2, 3, 4, 8, 12\}$. During the campaign every measured batch emits a checksum of C: among 222 configs and 1,110 measurements there is exactly one distinct checksum per n, i.e. all variants, thread counts and affinities produced identical matrices.

3.4 Environment

Table 2 summarises `data/results/environment.json`, captured during the measurement session. The High-performance power plan was active for the whole campaign and Balanced was restored afterwards.

Item	Value
CPU	Intel Core i7-1355U hybrid: 2 P-cores (2 threads each, CPU0–3) + 8 E-cores (1 thread each, CPU4–11); 10 physical, 12 logical CPUs
RAM / OS	16.79 GB; Windows 10 (10.0.26200)
Compiler	GCC 16.1.0 (MinGW-w64 ucrt64); `-O2 -march=native`
Native ISA	AVX, AVX2, FMA, SSE4.2 enabled; AVX-512 disabled
OpenMP	GCC libgomp; team fixed per run; default active spin-waiting
Power plan	High performance (GUID `8c5e7fda-...`) during campaign
Python	3.11.9, numpy 2.2.6, psutil 7.1.0

Table 2: Execution environment. Clock read-outs from `psutil` were unreliable on this platform and are not quoted.

3.5 Measurement protocol

Each configuration (variant $\times$ p $\times$ n $\times$ affinity) follows the protocol of Assignment 2:

1. **Probe:** one unchecked warm-up plus two timed single multiplications; a config is skipped if a probe exceeds 60 s. The bench budget is $\min(60\,\text{s}, 2 \times \max(\text{probe}, \text{probe warm-up}))$; the warm-up term covers the cost of creating the thread team, which the bench process pays again.

2. **Bench:** one warm-up, batch doubling until a batch lasts ≥ 0.2 s, then 5 measured repetitions of a full multiplication (kernel time only: loads and checksums are outside the timed region; `memset` of C is inside, as part of a complete multiplication).

3. **Summary statistic:** the **median** of the 5 repetitions; variability is the coefficient of variation (CV). Memory is the peak resident set size of the process. Budgets: 60 s and 4 GiB per child; every run is stored as JSON lines in `data/results/raw_detail/`.

Thermal drift control. A pilot campaign showed that results moved between sessions on this laptop: the same binary and config measured 3.22 s and 4.35 s (+35 %) in two sessions, and $S(1)$ fell to 0.66–0.76. The reported session therefore (1) warms up the machine for about

2.5 min with sustained 12-thread multiplications, and (2) *brackets* every thread sweep: for each n the sequence is `seqpre` $\rightarrow p \in \{1, \ldots, 16\} \rightarrow$ `seqpost`, and $T_{seq}(n)$ is the median of the 10 bracketed sequential runs, measured seconds apart from the sweep. The `simd2` and `pcore2` blocks run in the same session, one after the other. All 96 stable-session configs and all 126 earlier configs finished with status `ok`; the earlier rows stay in the CSVs for transparency, and the report quotes only the stable session.

4 Results

4.1 Runtime, speedup and efficiency

Table 3 gives the median times behind every S and E; Table 4 collects $S(p)$ and $E(p)$. The median CV of the 36 scaling configs is 5.1 % (max 13.6 %). Figures 1 to 3 show the same data.

		parallel time $T_p(n)$ (ms)					
n	T_{seq} (ms)	p=1	p=2	p=4	p=8	p=12	p=16
50	0.018	0.022	0.045	0.057	0.089	0.097	0.124
100	0.159	0.173	0.160	0.160	0.160	0.216	0.194
200	1.408	1.313	0.917	0.790	0.653	0.660	0.648
500	21.488	21.982	14.267	12.701	10.841	10.410	10.117
1000	233.654	235.686	167.644	124.109	119.753	91.202	92.933
2000	3626.93	3906.35	2646.38	1916.77	1490.82	1412.58	1398.74

Table 3: Median kernel time per multiplication (ms), median of five batched repetitions; T_{seq} = median of the 10 bracketed sequential runs.

n	p=1	p=2	p=4	p=8	p=12	p=16
50	0.79 (0.79)	0.39 (0.20)	0.31 (0.08)	0.20 (0.03)	0.18 (0.02)	0.14 (0.01)
100	0.92 (0.92)	1.00 (0.50)	1.00 (0.25)	1.00 (0.13)	0.74 (0.06)	0.82 (0.05)
200	1.07 (1.07)	1.54 (0.77)	1.78 (0.45)	2.16 (0.27)	2.13 (0.18)	2.17 (0.14)
500	0.98 (0.98)	1.51 (0.75)	1.69 (0.42)	1.98 (0.25)	2.06 (0.17)	2.12 (0.13)
1000	0.99 (0.99)	1.39 (0.70)	1.88 (0.47)	1.95 (0.24)	2.56 (0.21)	2.51 (0.16)
2000	0.93 (0.93)	1.37 (0.69)	1.89 (0.47)	2.43 (0.30)	2.57 (0.21)	2.59 (0.16)

Table 4: Speedup $S(p)$ with efficiency $E(p)$ in parentheses.

When does parallelism help? Only for $n \geq 200$. At $n = 50$ every parallel run is slower than sequential. At $n = 100$ the parallel time stays flat at ≈ 0.16 ms for $p \leq 8$: the fixed cost of the parallel region ($\sim$15–30 μs per multiplication at this size) dominates, and extra threads do not shorten it. For $n \geq 200$ the time falls up to $p = 8$ and then saturates. The best speedups are $S(12) = 2.56$ at $n = 1000$ and $S(16) = 2.59$ at $n = 2000$; E is at most 0.77 everywhere (best at $p = 2$ for $n \geq 200$). No configuration is superlinear. The single point above parity at p=1, $S(1) = 1.07$ at $n = 200$, lies inside that config's 8.2 % CV and is not claimed as a real gain.

4.2 Framework overhead with one thread

Table 5 confirms H2 where it matters: at $n = 50$ the parallel code with one thread is 26 % slower, which is about 4.5 μs of fork/join overhead on a kernel of only 17.5 μs. For $n \geq 200$ the ratio stays within ± 8 % of parity (both binaries run the same vectorised inner loop); the +8 % at $n = 2000$ comes from small drift inside the session between the baseline and the sweep.

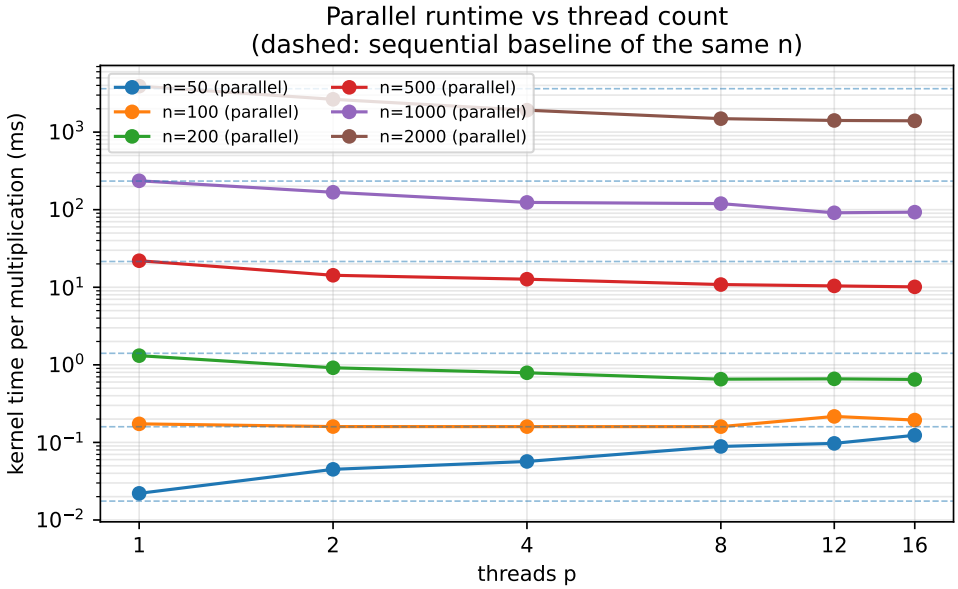


Figure 1: Kernel time vs thread count (log scale); dashed lines are the bracketed sequential baselines.

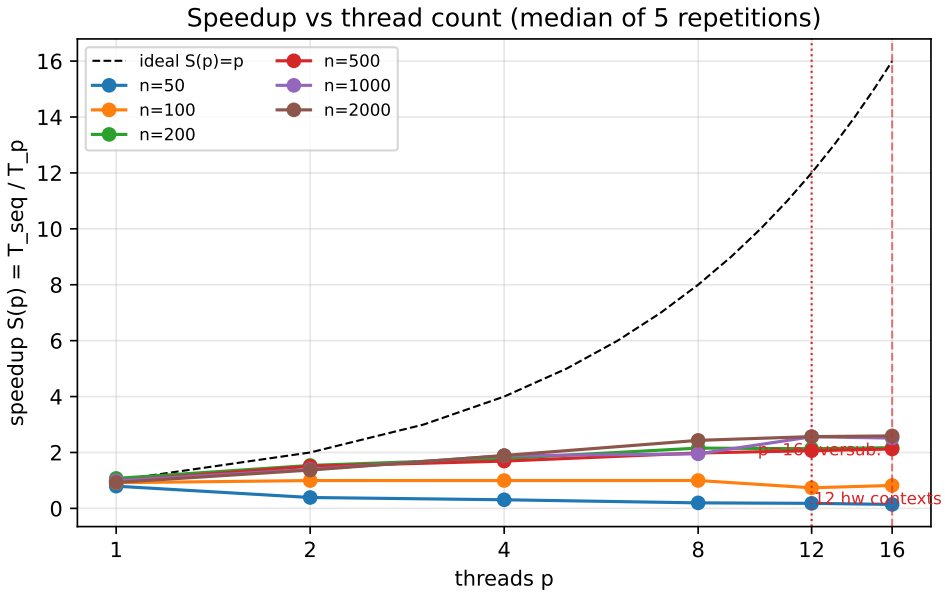


Figure 2: Speedup vs thread count against the ideal $S(p) = p$. The dotted line marks the 12 hardware contexts, the dashed line the oversubscribed $p = 16$.

n	T_{seq} (μs)	$T_p(1)$ (μs)	$T_p(1)/T_{seq}$
50	17.52	22.05	1.26
100	159.27	173.49	1.09
200	1408.39	1313.17	0.93
500	21488.46	21981.88	1.02
1000	233653.80	235686.20	1.01
2000	3626931.85	3906349.60	1.08

Table 5: Parallel implementation with one thread vs the sequential baseline (hypothesis H2).

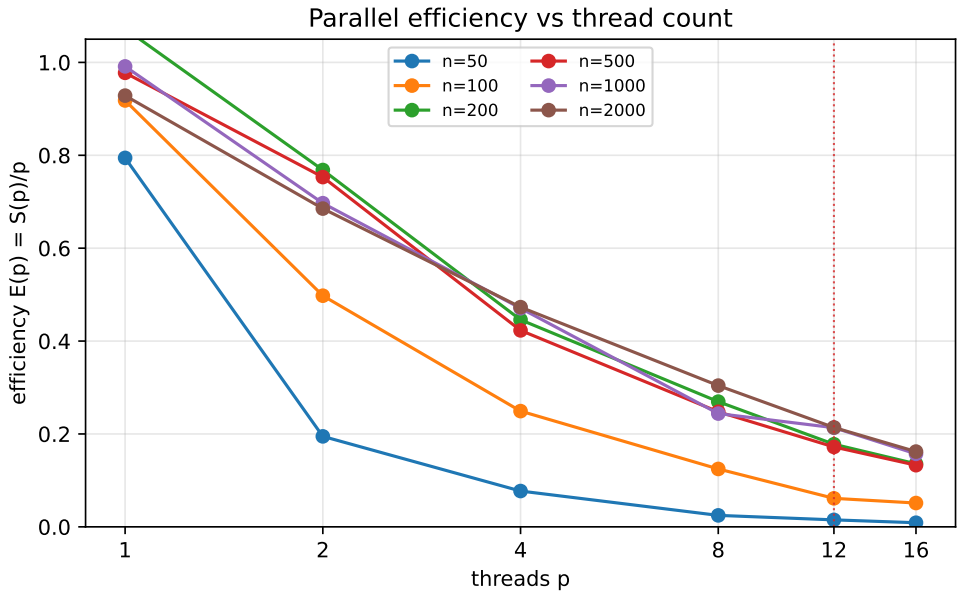


Figure 3: Efficiency vs thread count.

4.3 Vectorisation (optional extension)

Thread count is fixed and only the build changes (the `simd2` block, three variants measured back-to-back). Evidence that vectorisation happens: `-fopt-info-vec-optimized` reports the inner loop of `matmul3_seq` and of the pragma build vectorised with 32-byte vectors, and `objdump` finds packed FMA instructions in `matmul_ikj_seq` and in `matmul_ikj_par._omp_fn.0` of the `simd` binary (2 each) and none in the `scalar` binary. The `auto` binary has packed instructions only in the sequential function it never calls: **GCC did not vectorise the parallel loop under `-O2`, so the explicit pragma was necessary** (the caution in H4). Full listings are in `data/results/vectorisation.txt`.

	time at $p = 1$ (ms)			scalar/simd gain		
n	scalar	auto	simd	p=1	auto/simd	p=8
200	3.46	3.47	1.12	3.10	3.11	1.79
500	60.80	50.79	24.54	2.48	2.07	1.83
1000	506.99	502.37	254.34	1.99	1.98	1.64
2000	5080.69	5205.41	3538.06	1.44	1.47	1.47

Table 6: Vectorisation control at fixed thread count (`simd2` block). `auto` tracks `scalar` because the parallel loop was not vectorised by default.

Table 6 and Figure 4 quantify H4. The pragma is worth 3.10× at $n = 200$ but only 1.44× at $n = 2000$ ($p = 1$). At $n = 200$ the working set fits in cache, so the run is compute-bound and close to the 4× width of 256-bit vectors over scalar code; at $n = 2000$ the three 32 MB matrices are streamed from DRAM, and arithmetic is no longer the bottleneck. The gain remains at high thread counts (1.47–1.83× at $p = 8$): vectorisation and threads compose. Their effects stay separate by construction: every row of Table 4 uses the `simd` binary, while Table 6 varies only the binary at fixed p.

4.4 Oversubscription at $p = 16$

The hardware has 12 execution contexts, so $p = 16$ is an oversubscribed configuration (H3). At $n = 2000$, $p = 16$ is inside the variability of $p = 12$ (S: 2.59 vs. 2.57, i.e. +0.8 %, below the

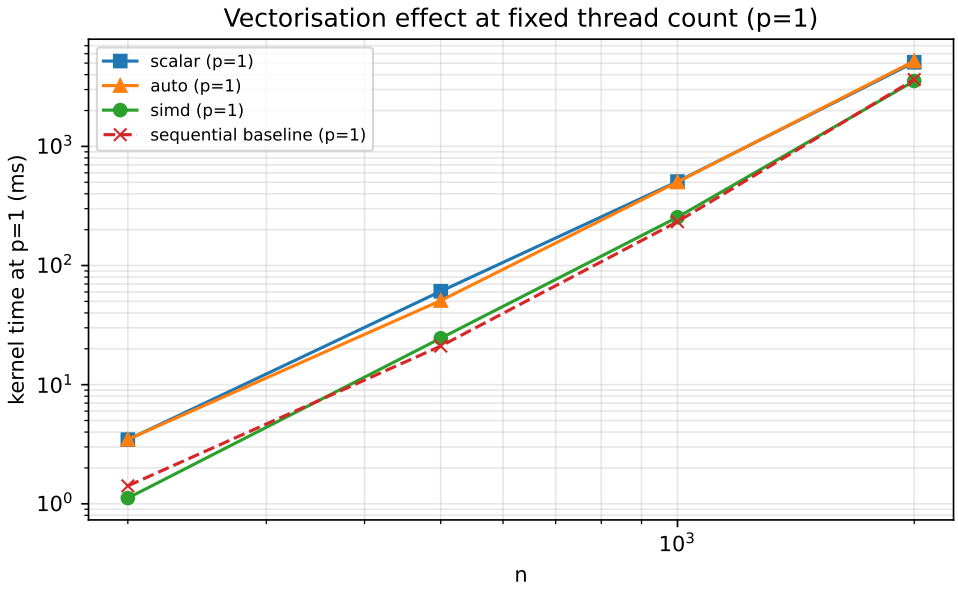


Figure 4: Vectorisation effect at $p = 1$; the sequential baseline uses the same vectorised inner loop as `simd`.

3.7–3.9 % CV of both configs): no gain, but no collapse either. At $n = 1000$ it is 2 % slower; at $n = 100$, 10 % slower than $p = 12$; at $n = 50$, 28 % slower: sixteen thread wake-ups and barrier crossings are paid for a kernel that is shorter than the parallel region itself. Efficiency at $p = 16$ never exceeds 0.16. Oversubscription is therefore pointless: the extra threads only compete for the same 12 contexts and add scheduling cost exactly where the useful work is smallest.

4.5 P-cores vs. all cores

n	p	all CPUs (ms)	P-cores only (ms)	ratio
500	1	21.98	22.05	1.00
500	2	14.27	15.46	1.08
500	4	12.70	13.72	1.08
1000	1	235.69	266.62	1.13
1000	2	167.64	165.37	0.99
1000	4	124.11	113.19	0.91
2000	1	3906.35	3443.15	0.88
2000	2	2646.38	2280.18	0.86
2000	4	1916.77	1755.45	0.92

Table 7: Same binary and p with the process confined to CPU0–3 (P-core threads) vs. CPU0–11 (ratio < 1 favours the P-cores-only run).

At $n = 2000$ (Table 7, Figure 5) confining the process to the four P-core threads is 12–14 % faster than scheduling over all 12 contexts: the P-cores do more work per cycle, and no E-core thread becomes the slowest partition behind the barrier. At $n = 1000$ the effect changes sign with p, and at $n = 500$ the restricted run is 8 % slower; both are inside the ±13 % variability of these medium configs. The advantage is therefore a large-n result.

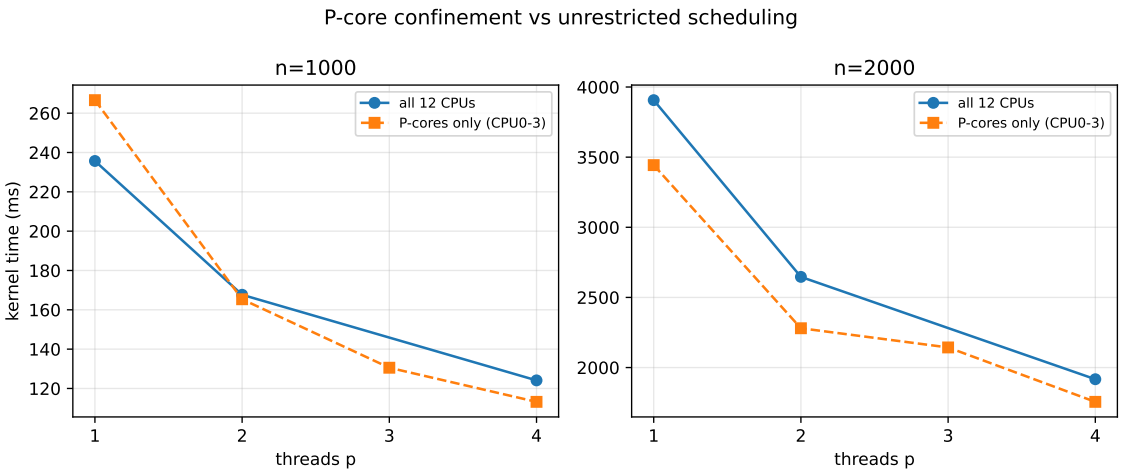


Figure 5: P-core confinement vs. unrestricted scheduling.

4.6 Memory

Peak resident memory is 5.4, 6.2, 11.0, 28.2 and 96.8 MB for $n = 50, 200, 500, 1000, 2000$. At $n = 2000$ this is essentially 3×32 MB of matrices plus process overhead, and it does not depend on p: static row partitioning allocates no per-thread copies and creates no races (consistent with the checksum invariance).

5 Discussion

Answer to the research question. On this machine and kernel, parallel execution helps only from $n = 200$ upward and only up to about $p = 8$–12; the measured ceiling is $S \approx 2.6$, far below $p = 12$. Three effects add up: (1) fixed parallel-region costs, which are 26 % of an $n = 50$ multiplication and explain the flat $n = 100$ curve; (2) core heterogeneity: an equal row split is not equal work on 2 P-cores + 8 E-cores, and the P-cores-only run beating all-CPU scheduling by up to 14 % at $n = 2000$ shows that placement matters at large n; and (3) memory: at $n = 2000$ the streaming of three 32 MB matrices limits useful parallelism and also caps the vectorisation gain at 1.44×. H1–H4 are all supported, and H4's second part (default vectorisation cannot be assumed) is demonstrated per binary.

Methodological findings. Two instrument bugs were found and fixed before the reported session: a gate that ignored the thread-team creation cost of the bench process (fixed by including the probe warm-up in the gate), and cross-phase thermal drift that depressed $S(1)$ to 0.66 (fixed by warm-up and sequential bracketing). The pilot rows stay in the CSVs. The stability of the reported session is visible as $S(1) \in [0.92, 1.07]$ for $n \geq 100$ and median CV 5.1 %.

Limitations.

- One hybrid laptop, one kernel, one input distribution: results do not generalise to servers, other BLAS libraries or structured data.
- Windows places threads inside the affinity mask; P/E placement and boost states were not pinned per thread, so ±5–13 % run-to-run variability remains (median CV 5.1 %).
- At $p = 1$ a residual ±8 % gap remains between baseline and parallel binary (drift, scheduling), so $S(1)$ is parity within noise, not an exact 1.00.
- No hardware performance counters were collected on Windows, so efficiency losses are not split into bandwidth and other terms.

- The missing vectorisation of the `auto` build is a GCC 16 `-O2` cost-model decision; other compilers or `-O3` may differ.

As the assignment requests, the claims are limited to these implementations, inputs and hardware [6, 7].

6 Conclusions

Within the tested scope:

1. The OpenMP row-parallel `ikj` kernel beats its sequential baseline only for $n \geq 200$, with a peak of $S(12) = 2.56$ ($n = 1000$) and $S(16) = 2.59$ ($n = 2000$): threads buy at most about 2.6× on 12 hardware contexts.
2. Efficiency is the binding metric: $E(p) \leq 0.77$ everywhere and ≤ 0.21 for $p \geq 12$, because of fixed parallel-region cost at small n and of P/E asymmetry plus memory bandwidth at large n.
3. The one-thread parallel overhead is 26 % at $n = 50$ and within ±8 % of parity for $n \geq 200$.
4. Oversubscription ($p = 16$ on 12 contexts) gives no gain at large n and costs up to 28 % at $n = 50$: it hurts exactly where the work per thread is smallest.
5. Explicit `#pragma omp simd` is worth 1.4–3.1× at fixed p, was necessary because GCC did not vectorise the parallel loop by default, and composes with threading.
6. Confining the process to the four P-core threads is 12–14 % faster at $n = 2000$.

Open questions: whether a core-weighted row partition raises the scaling ceiling, whether tiling relieves the bandwidth limit, and how these results compare with a distributed implementation (Assignment 4).

7 References and materials

All source code, shared inputs, raw measurements (CSV per config, JSON lines per run), `environment.json`, `vectorisation.txt`, the executed analysis notebook and every figure are in the public repository `https://github.com/aledelgadoo/individual-assignments-bigdata`. The Git tag `assignment-3` marks the submitted state (tags `assignment-1` and `assignment-2` keep the earlier reports). `README.md` documents the build, input generation, self-test, campaign and analysis commands that regenerate this report end to end; the notebook `notebooks/03_analysis.ipynb` writes the summary tables (`summary_scale.csv`, `summary_simd.csv`, `summary_pcore.csv`) quoted here.

Generative-AI usage statement

Following the course policy, the author discloses that the AI assistant **opencode** (model `opencode/mimo-v2.6-flash-free`) was used, under the author's direction, for planning the OpenMP/SIMD design and the measurement protocol, drafting and debugging the C code and Python scripts, diagnosing the two pilot-campaign instrument bugs, generating the analysis notebook and figures, and revising the English of this report. The author reviewed all generated code, ran and inspected the self-tests, supervised every campaign execution, verified the numbers quoted here against the raw CSV files, and is responsible for the measurements, the conclusions and any errors.

References

[1] G. Strang, *Introduction to Linear Algebra*, 5th ed. Wellesley–Cambridge Press, 2016.

[2] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. MIT Press, 2022.

[3] OpenMP Architecture Review Board, *OpenMP Specification for Offload and Shared Memory, Version 5.2*, 2022. [Online]. Available: `https://www.openmp.org/specifications/`

[4] Intel Corporation, *Intel Hyper-Threading Technology: Software Optimization Guide* and Alder Lake performance/efficiency-core documentation. [Online]. Available: `https://www.intel.com/`

[5] Intel Corporation, *Intel Intrinsics Guide* (AVX2/FMA instruction reference). [Online]. Available: `https://www.intel.com/`

[6] W. P. Fleming and J. M. Wallace, "How not to lie with statistics: the correct way to summarize benchmark results," *Communications of the ACM*, vol. 29, no. 3, pp. 218–221, 1986.

[7] T. Mytkowicz, A. Diwan, M. W. Hauswirth, and P. F. Sweeney, *Measuring Computer Performance: A Practitioner's Guide*. Cambridge University Press, 2009.

[8] ISO/IEC 9899:2011, "Information technology — Programming languages — C," International Organization for Standardization, 2011.

[9] G. Rodola, *psutil: cross-platform process and system utilities*, 2026. [Online]. Available: `https://github.com/giampaolo/psutil`